

School of Science, Computing and Engineering Technologies
Swinburne University of Technology
SWE30003 Software Architectures and Design
Tutorial Week 4
(Software Quality Attributes)

Hotel Information System

1. Given the context of the hotel information system as discussed in Lecture 2 and Tutorial Week 3,

a. What are the quality requirements relevant to the *Check-In* and *Check-Out* tasks?

=> **Performance Efficiency (Time Behavior):** The system's response latency must remain minimal and stable under intense concurrent load to prevent queues from forming at the physical front desk.

=> **Availability & Reliability:** The system must guarantee near-zero downtime. A system crash during peak rush hours completely paralyzes the lobby operations and halts business.

=> **Usability (Operability & Learnability):** The user interface must be optimized for speed, supporting rapid keyboard shortcuts and clean data-entry fields to minimize human processing time per guest.

=> **Security (Data Protection & Privacy):** The system must handle sensitive data securely, including cryptographic encryption for personal identity documents (passport/ID scans) and financial data (credit card transactions).

b. What quality requirements are similar and what not, between these two tasks?

=> Similarities:

+ **High Performance Requirements:** Both tasks must absorb massive traffic spikes without degrading speed. Guests in both lines are highly sensitive to long waiting times.

+ **High Availability Needs:** A crash in either sub-system at 9:00 AM or 3:00 PM creates immediate operational chaos at the physical lobby desk.

=> Differents:

Dimension	Check-In	Check-Out
Guest Psychology	Guests have just traveled a long distance; they are exhausted and demand immediate room access to rest or prepare for urgent business meetings.	Guests are under intense time constraints. They are hurrying and anxious to catch departing flights, trains, or morning corporate schedules.
Data Flow Focus	Data Ingestion & Filtering (Mixed Read/Write): Querying a Booking ID, verifying that the Room State is <i>Cleaned</i> and <i>Inspected</i> , and capturing/saving new identity data.	Heavy Financial Transaction (Heavy Write Path): Instantly consolidating dynamic, multi-source <i>Incidental Expenses</i> (room service, laundry) into an immutable final invoice and settling payments.
Worst-Case Risk	Frustrated guests, bad initial impressions, and poor customer satisfaction reviews.	Guests missing their flights. This introduces severe financial and legal liabilities for the hotel's brand reputation.

c. Please specify each of these requirements, in a precise and verifiable manner.

=> Performance Efficiency - Specification: During peak operational hours (08:00 AM to 10:00 AM), the system must consolidate all dynamic incidental expenses and fully render the final invoice presentation screen within less than 2.0 seconds from the moment the front-desk clerk clicks 'Generate Bill', under a benchmarked concurrent load of 50 active workstation nodes

=> Availability - Specification: The Hotel Information System (HIS) must maintain a continuous operational availability of 99.9% (\$24/7/365\$), restricting unscheduled downtime to a maximum of 8.76 hours per calendar year, with zero allowed unscheduled downtime during daily peak windows (08:00 AM - 10:00 AM and 02:00 PM - 04:00 PM).

=> Usability - Specification: A trained front-desk operator must be able to successfully complete a standard Pre-Booked Check-In workflow (including identity data capture and keycard initialization) in fewer than 45 seconds using keyboard-only shortcuts, without requiring mouse interaction.

=> Security - Specification: All credit card data transactions processed during the Check-Out task must be encrypted in transit using TLS 1.3 and at rest using AES-256 encryption, achieving full compliance with PCI-DSS Level 1 security standards, verified by automated weekly vulnerability scans.

2. In general, considering all working areas, what would be the top 3 to 5 quality attributes applicable to the Hotel Information System?

=> External Quality:

+ Definition: Software characteristics directly experienced, observed, and felt by the end-user during runtime.

+ Example: An application that responds quickly (high performance) or has an intuitive, attractive user interface (high usability).

=> Internal Quality:

+ Definition: The structural health, code integrity, and architectural design of the source code, which is completely invisible to the end-user but vital to developers.

+ Example: A codebase with clean abstraction and low coupling (high modifiability), allowing developers to add features quickly without introducing bugs.

Further General Discussion Points

3. What are the general software quality attributes? Describe each of the software quality attributes.

=> Availability

- Description: The system's ability to remain operational, accessible, and fully functional to users during its scheduled uptime, resisting or recovering from system faults.
- Key Metric: Uptime percentage (e.g., 99.9% availability).

=>Modifiability

- Description: The ease with which a software system can be altered, extended, updated, or refactored to accommodate new business requirements, fix bugs, or improve performance.
- Key Metric: Cost and time required to implement a code change.

=> Performance

- Description: The system's responsiveness, speed, throughput, and resource utilization when executing operations under specified benchmarked workloads.
- Key Metric: Latency (response time in milliseconds) and throughput (requests per second).

=> Security

- Description: The system's capacity to protect sensitive data, resist unauthorized malicious attacks, and guarantee proper access control and identity validation.
- Key Metric: Number of vulnerabilities discovered or time taken to detect and contain a breach.

=> Testability

- **Description:** The ease with which software components can be isolated, executed, and validated to demonstrate their correctness and uncover hidden defects.
- **Key Metric:** Percentage of code coverage by automated tests.

=> Usability

- **Description:** The degree of ease, learnability, operational efficiency, and aesthetic satisfaction an end-user experiences when interacting with the application's interface.
- **Key Metric:** Task completion rate and time taken for a user to complete a standard workflow.

4. What are some forms of scalability? How do they work?

=> “Scaling Up”

- + **The Concept:** Making a single, existing server bigger, faster, and more powerful to handle increased loads.
- + **The Action:** You keep the exact same machine or virtual instance, but you physically upgrade its hardware—adding more CPU cores, increasing the RAM capacity, or swapping to faster Solid-State Drives (SSDs).
- + **The Analogy:** If your single delivery van is overflowing with packages, you sell it and buy a massive, heavy-duty commercial cargo truck instead.
- + **Physical Hardware Ceiling:** You eventually hit a maximum hardware boundary dictated by motherboard limitations. Once you buy the strongest machine available on the market, you cannot scale any further.
- + **Downtime Risks:** Upgrading physical components or resizing a virtual server instance often requires a system restart, causing temporary downtime problems.

=> “Scaling Out”

- + **The Concept:** Adding more servers to your existing pool of resources, rather than making a single server stronger.
- + **The Action:** Instead of one giant machine, you connect multiple standard, smaller servers together in a clustered infrastructure.
- + **Traffic Routing:** You place a **Load Balancer** in front of the server pool. The load balancer acts as a traffic cop, distributing incoming user requests evenly across all available active nodes.
- + **The Analogy:** Instead of relying on one massive cargo truck, you deploy a fleet of 50 standard delivery vans that all work in parallel to distribute the load.
- + **Virtually Infinite Headroom:** There is no hard hardware ceiling. If incoming traffic doubles or triples, you simply spawn more standard server instances into the pool seamlessly with zero system downtime.
- + **High Flexibility & Cost-Efficiency:** It allows for Elastic Auto-Scaling (especially in cloud environments like AWS or Google Cloud). The system automatically provisions more servers during sudden flash surges (like

holiday rushes or peak operational windows) and shrinks the pool back down when traffic subsides, ensuring you only pay for what you use.

5. What is Throughput? Compare and contrast average and peak throughput.

=> Definition: The number of successful actions, transactions, or user requests a software system can reliably process within a specific timeframe.

=> Measurement: Typically measured in Requests Per Second (RPS) or Queries Per Second (QPS).

Compare and Contrast		
Feature	Average Throughput	Peak Throughput
Definition	The total volume of traffic received over a long period divided evenly down to the second.	The absolute maximum traffic intensity experienced during a sudden, concentrated surge.
Represents	Steady, everyday baseline traffic.	The highest volume of concurrent requests hitting servers at a single point in time.
Design Risk	Designing for this causes systems to crash during sudden high-demand events.	Designing for this ensures infrastructure can handle worst-case load spikes.

6. How does the quality attribute of security affect availability in terms of the authentication and the authorization process?

=> The Authentication (AuthN) Process

- Cryptographic Overhead: Verifying identities using 12-character rotating passwords and biometric scans requires CPU-intensive encryption checks, pattern-matching, and cryptographic hashing.
- Resource Exhaustion: Implementing aggressive security rules—like a short 3-minute inactivity timeout—forces massive numbers of users to repeatedly re-authenticate at the same time. This floods the auth-servers, creating a self-inflicted DDoS effect that drains system memory pools and takes the application offline.

=> The Authorization (AuthZ) Process

- Continuous Check Bottlenecks: Validating user permissions (e.g., checking role-based access control or demanding manager overrides for specific commands) forces the system to run continuous database lookup loops.
- Process Paralysis: The operational friction of constantly verifying rights creates human bottlenecks. Users face severe delays waiting for access

clearance, which effectively lowers the availability of the overall business service to the end client.

7. Software qualities can be categorised into two (2) broad categories: external and internal. Please explain both, including an example.

=> **External:** External software qualities refer to the characteristics of a system that are directly experienced, observed, and felt by the end-users during runtime execution. Furthermore, these attributes measure how effectively the application behaves from an operational and user-facing perspective.

=> **Internal:** Internal software qualities refer to the structural health, code integrity, and architectural design of the source code itself. These characteristics are completely invisible to the end-user during runtime, but they are heavily relied upon by the development and engineering teams who must maintain, update, and test the software behind the scenes.

8. Any questions from weekly submissions? => Maybe not